

CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 09 · LECTURE NOTES

Netcode I: The Latency Problem

Distributed systems with a 16 millisecond deadline.

MODULE OUTCOMES

- Compare netcode architectures and their failure modes.
- Explain the authority question and why it decides everything.
- Explain deterministic lockstep and its requirements.
- Explain state synchronisation and its bandwidth problem.
- Choose an architecture from the game's requirements.

1 The constraint

THREE EXTRA CONSTRAINTS THAT CHANGE EVERYTHING

A hard deadline. 16.7 ms at 60 Hz, and a frame that misses it is visible to the player as a stutter. **There is no option to wait** — which removes every technique in Modules 04 through 08 that involves a round trip before acting.

An adversarial participant. Every client runs on hardware the attacker physically owns, with a debugger attached and the memory readable and writable. **Byzantine by default** (Module 01 §3), and not occasionally but as the normal case.

And continuous state. Not a log of discrete operations to be agreed upon but positions, velocities, and orientations changing every single tick — which is a poor fit for the replicated-log abstraction that Module 06 reduces everything to.

So consensus is out (Module 06 §4, on the arithmetic), **quorum replication is out** (no time for the round trip), and **CRDTs are out** (Module 08 §3 — a game needs a referee, and 'who shot first' is a decision rather than a merge). **What remains is an authority and a great deal of educated guessing**, which is the subject of this module and the next two.

2 Architectures

Architecture	Who has authority	The trade
Authoritative dedicated server	The server, entirely. Clients send inputs and receive state.	Cheat-resistant (Module 11), costs server capacity , and introduces a round trip of input latency unless the client predicts (Module 10).
Listen server	One player's machine, which also plays.	Free to operate , and that player has zero latency and perfect information while everyone else does not . Structurally unfair, and common in casual games for cost reasons.
Deterministic lockstep	Everyone, by construction — all clients compute the same world from the same inputs.	Minimal bandwidth , permanent input delay , and catastrophic failure on desync (§3).
Peer-to-peer with trusted clients	Each client is authoritative over its own objects.	Trivially cheatable — a client simply declares it did not take damage. Acceptable for co-operative play only.
Rollback (GGPO-style)	Lockstep, with local prediction and re-simulation.	Excellent responsiveness , and requires a simulation cheap enough to re-run several ticks within one frame (Module 10 §4).

THE AUTHORITY QUESTION COMES FIRST

Who decides whether the shot hit? Whoever decides is the authority; everyone else is displaying a guess. Answering this one question determines the bandwidth profile, the cheat resistance, and how the game feels.

If the client decides, it can lie, and in any game with competition it will (Module 11). **No cryptography fixes this**, because the client legitimately possesses every key and every piece of code it would need to produce a perfectly well-formed false claim. This is a genuinely unsolvable problem, not a hard one.

If the server decides, the player must wait a full round trip to see the result of their own action — which at 100 ms is unplayable for anything requiring aim or timing, **unless the client predicts** (Module 10).

So the answer for any competitive game is server authority plus client prediction, and the whole of Module 10 is about making that combination feel instantaneous while remaining server-authoritative. **It is one of the more elegant pieces of engineering in this program**, and it is far less widely understood than it deserves to be.

3 Deterministic lockstep

Send only INPUTS. Every client runs the identical simulation.

```
tick N:  gather local input
         broadcast it
         WAIT for every other player's input for tick N
         simulate tick N with all inputs
         -> every client computes an identical world
```

BANDWIDTH: a few bytes per player per tick, INDEPENDENT of how many entities exist. 2000 units costs the same as 5. This is why RTS games use it.

PROBLEM 1 -- INPUT DELAY

cannot simulate tick N until everyone's input arrives, so everyone waits for the SLOWEST player.
Mitigation: simulate tick N using inputs gathered at N-3. That is three ticks of input lag, permanently.

PROBLEM 2 -- DESYNC

determinism must be EXACT and bit-identical. Float differences across compilers or CPUs, map iteration order, uninitialised memory, or one unseeded random call diverge the simulations -- and the divergence COMPOUNDS from that tick onward. Detect with a periodic state checksum. Recovery is resynchronisation or dropping the player.

This is exactly Module 06's replicated state machine with the consensus layer removed — the same reduction (agree on the inputs, derive the state), the same absolute determinism requirement, and the same failure mode when it is violated. **The compounding is what makes desync catastrophic rather than cosmetic:** one unit's position differing by one bit at tick 1000 changes what it collides with at tick 1001, which changes the whole battle by tick 1100. There is no partial recovery.

4 State synchronisation

THE SERVER SENDS WHAT IT DECIDED

The server runs the authoritative simulation and sends snapshots of world state to each client. Clients display what they are told and send only their inputs back.

There is no determinism requirement at all. Clients need not agree with one another or reproduce the server's arithmetic — they only need to receive and render. **This removes the entire desync failure mode,** which is a very large practical advantage.

And there is no waiting. A client that misses a snapshot simply uses the next one; packet loss degrades smoothly into slightly staler information rather than stalling everyone (Module 02 §1).

The cost is bandwidth, which scales with the amount of state rather than with the number of players. Which is exactly why shooters use state sync and strategy games use lockstep: a shooter has dozens of relevant entities and a dozen players, while an RTS has thousands of units and four players. **The architectures are not better or worse; they are matched to opposite ratios.**

- **Relevance or interest management.** Send each client only what it could plausibly perceive — what is nearby, visible, or audible. **By far the largest single saving,** often an order of magnitude, and it doubles as an anti-cheat measure (Module 11 §3) since information not sent cannot be extracted.
- **Delta compression** against the last acknowledged snapshot — transmit only the fields that changed since the client last confirmed receipt.
- **Quantisation.** A position in a bounded world does not need three 32-bit floats; 16 bits per axis over the world extent is visually indistinguishable and halves the cost. The same applies to orientations (smallest-three quaternion encoding) and velocities.
- **Prioritisation.** Nearby, recently-changed, and player-relevant objects update every tick; distant scenery updates rarely. A per-object priority accumulator is the usual mechanism.
- **And all of it rides unreliable UDP** (Module 02 §1), so a lost snapshot costs nothing — the next one supersedes it. **Which is precisely why delta compression must be against an *acknowledged* baseline rather than the last one sent:** if the client never received the baseline, every subsequent delta is meaningless, and the error persists forever rather than for one frame. **This is the subtle bug in naive implementations,** and it only appears under packet loss, which is why localhost testing misses it entirely.

CHOOSING AN ARCHITECTURE

Few entities, many players, competitive: state synchronisation with an authoritative dedicated server. Shooters, battle royales, and MMOs. The bandwidth is affordable because relevance filtering is effective, and the cheat resistance is essential.

Many entities, few players, bandwidth-constrained: deterministic lockstep. Real-time strategy, and anything where the unit count makes state sync impossible. Accept the input delay and invest heavily in determinism testing.

Two players, frame-perfect input matters: rollback netcode. Fighting games — and GGPO genuinely changed the genre, to the point that rollback support is now a purchasing consideration for players.

Co-operative only, where trust is acceptable: peer-to-peer with client authority. **And be honest in the design document that it is cheatable** — which is entirely fine when the only person who can be cheated is a friend who chose to play with you, and is not fine the moment a leaderboard exists.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Glenn Fiedler — Networked Physics and the Gaffer On Games series (free)	The reference for this module and the next two. Start with 'What Every Programmer Needs To Know About Game Networking'.
Valve — Source Multiplayer Networking (free)	A shipping architecture documented by its authors — snapshots, interpolation, prediction, and lag compensation in one coherent description.
Bettner & Terrano — 1500 Archers on a 28.8: Network Programming in Age of Empires (free)	The classic lockstep paper , and still the clearest account of §3's trade-offs and desync hunting.
Ruoyu Sun / GGPO — rollback networking documentation (free)	The rollback architecture, with source.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Measure your own input-to-display latency** locally, then add 100 ms of network delay and measure the difference in feel.
- 2 Implement a trivial two-player simulation with client authority and **cheat in it** to show how easy it is.
- 3 Implement deterministic lockstep for a simple simulation.

- 4 **Add a state checksum** and deliberately introduce a desync (an unseeded random call). Measure how many ticks before the worlds visibly differ.
- 5 **Measure lockstep bandwidth against entity count** and confirm it is flat.
- 6 Implement state synchronisation with full snapshots and measure the bandwidth.
- 7 **Add relevance filtering** and measure the saving.
- 8 Add delta compression against an acknowledged baseline.
- 9 **Then delta against the last snapshot sent instead**, introduce 5% loss, and demonstrate the permanent corruption.
- 10 Quantise positions to 16 bits per axis and verify the error is below a pixel at typical viewing distance.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What three constraints does a game add, and which techniques do they rule out?
- 2 Compare five netcode architectures on authority and trade.
- 3 Why does the authority question come first?
- 4 Why can cryptography not make a client trustworthy?
- 5 Describe lockstep and give its two problems.
- 6 Why does desync compound?
- 7 What does state sync give up and what does it gain?
- 8 Name five bandwidth reductions for state sync.
- 9 Why must delta compression use an acknowledged baseline?